ASSIGNMENT 20

dockerfile WORKDIR and Docker network

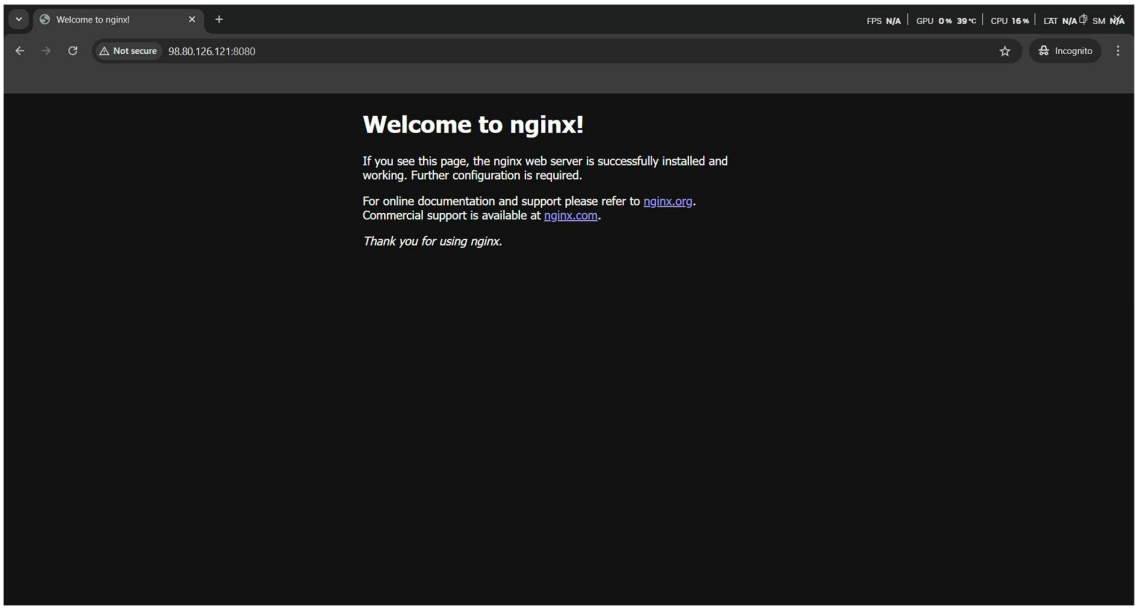
Creation of NGINX and HTTPD Using Docker on AWS Linux Instance

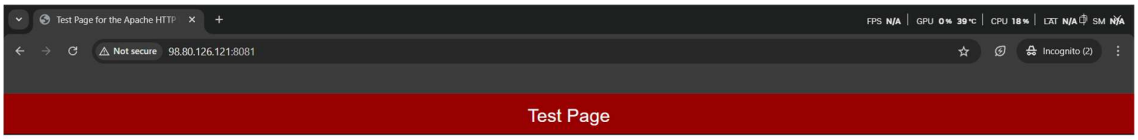
In this task, I created NGINX and HTTPD web servers using Docker on an AWS Linux EC2 instance. First, an AWS EC2 Linux instance was launched and Docker was installed and started to enable containerization. Docker is used to package applications along with their dependencies, making deployment easy and consistent.

After installing Docker, I created application files inside the instance, such as an index.html file containing custom web page content. Then, a **Dockerfile** was created using a text editor. In the Dockerfile, a base image (NGINX or HTTPD) was specified, and the index.html file was copied into the web server's default directory. The required port was exposed so the application could be accessed through a browser.

Next, the Docker image was built using the docker build command. This image contained the complete configuration of the web server along with the application files. Once the image was successfully created, a Docker container was launched using the docker run command with port mapping. This allowed the container's web server port to be accessed from the EC2 instance public IP.

Finally, the running container was verified using Docker commands, and the web application was accessed from a browser using the EC2 public IP and mapped port. In this way, NGINX and HTTPD web servers were successfully created and deployed using Docker on an AWS Linux instance.





Test Page

This page is used to test the proper operation of the Apache HTTP server after it has been installed. If you can read this page, it means that the Apache HTTP server installed at this site is working properly.

If you are a member of the general public:

The fact that you are seeing this page indicates that the website you just visited is either experiencing problems, or is undergoing routine maintenance.

If you would like to let the administrators of this website know that you've seen this page instead of the page you expected, you should send them e-mail. In general, mail sent to the name "webmaster" and directed to the website's domain should reach the appropriate person.

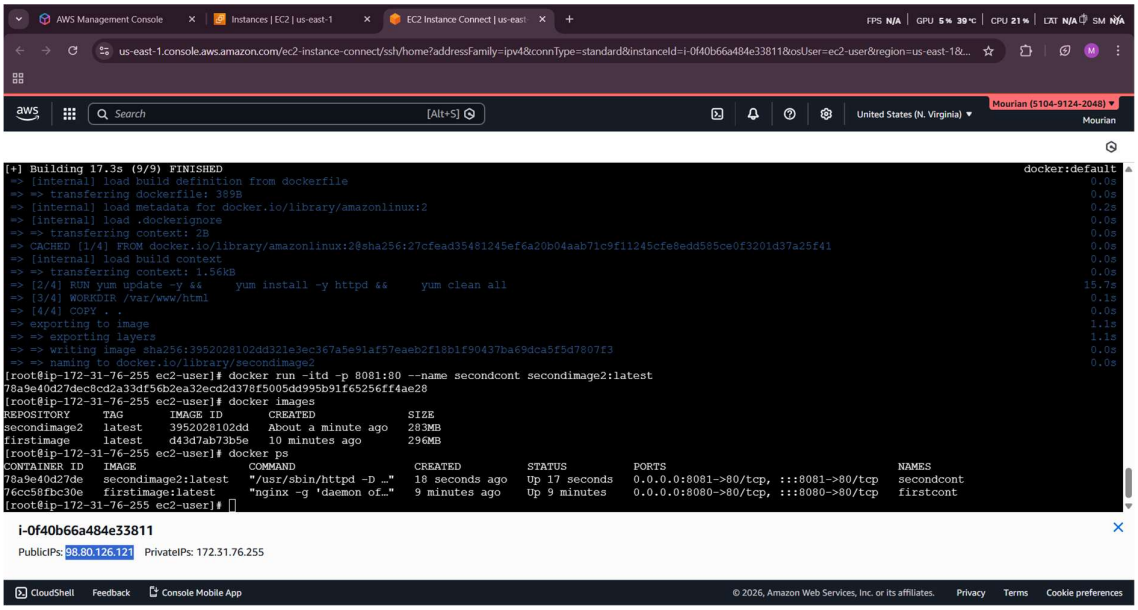
For example, if you experienced problems while visiting www.example.com, you should send e-mail to "webmaster@example.com".

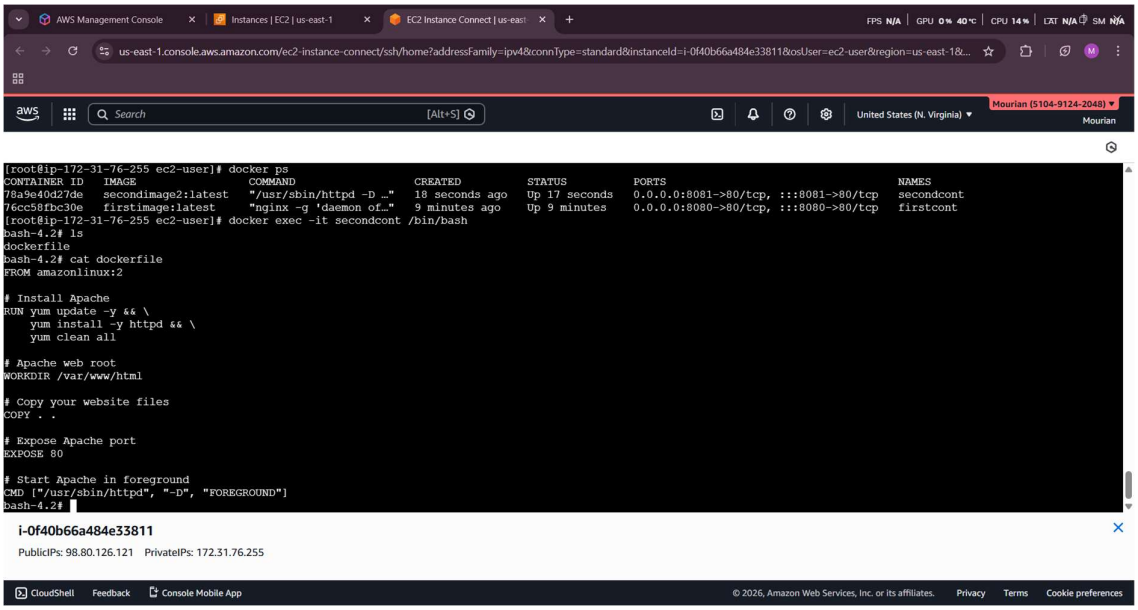
If you are the website administrator:

You may now add content to the directory `/var/www/html/`. Note that until you do so, people visiting your website will see this page, and not your content. To prevent this page from ever being used, follow the instructions in the file `/etc/httpd/conf.d/welcome.conf`.

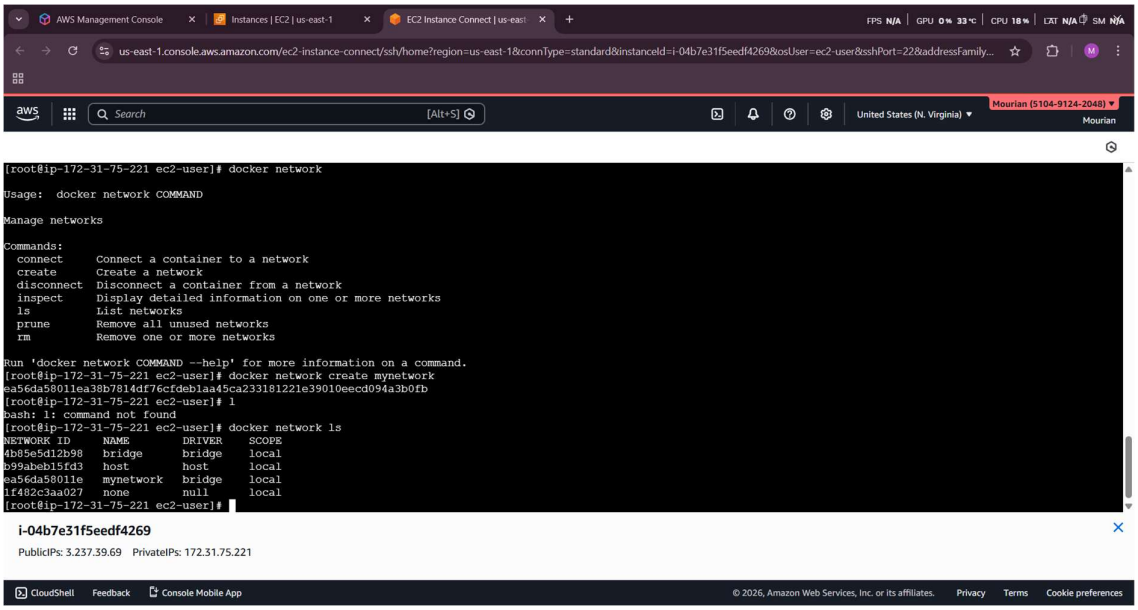
You are free to use the image below on web sites powered by the Apache HTTP Server:

Powered by APACHE 2.4





DOCKER NETWOKS:



Docker networking is used to enable communication between Docker containers, the Docker host, and external networks. Docker provides built-in network drivers that allow containers to connect securely and efficiently without manual network configuration. Each container is attached to a network and is assigned an IP address automatically by Docker.

Docker supports different types of networks such as **Bridge**, **Host**, **None**, and **Overlay**. The **Bridge network** is the default network type and is commonly used when multiple containers need to communicate on the same host. Containers on the same bridge network can communicate with each other using container names. **Host network** removes network isolation and allows the container to use the host's network directly, which improves performance but reduces security. **None network** disables networking completely for the container, providing full isolation.

Docker also supports **Overlay networks**, which are mainly used in Docker Swarm. Overlay networks allow containers running on different Docker hosts to communicate with each other as if they were on the same network. Docker automatically manages routing and IP assignment in overlay networks, making them suitable for distributed applications.

Overall, Docker networking simplifies container communication, improves application isolation, and provides flexible network management. It plays a crucial role in deploying scalable and secure containerized applications.